Post-Improvement of the Capacitated Vehicle Routing Problem Using a Weighted Partial Max-SAT Solver

Complete Technical Documentation

RTSS Research Group

March 16, 2026

Contents

1 Introduction

1.1 Problem Overview

The Capacitated Vehicle Routing Problem (CVRP) is one of the most studied combinatorial optimization problems in operations research. Given a set of customers with known demands, a central depot, and a fleet of homogeneous vehicles with limited capacity, the objective is to find a set of routes — each starting and ending at the depot — that serves all customers while minimizing the total travel distance.

CVRP is NP-hard, meaning that no polynomial-time algorithm is known to solve all instances optimally. Practical approaches therefore combine heuristic construction with exact or near-exact improvement methods.

1.2 Our Approach

We propose a **post-improvement pipeline** consisting of four phases:

Phase 1: Initial Solution Construction — An enhanced Clarke-Wright (CW) savings algorithm with a multi-strategy route reduction mechanism and sweep fallback

Phase 2: Intra-Route Improvement — 2-Opt local search applied to each individual route

Phase 3: Search Space Expansion — K-Nearest Neighbors (KNN) using a Minimum Spanning Tree to define the set of candidate edges for each vehicle

Phase 4: Global Optimization — A Weighted Partial Max-SAT model solved by the `clasp` PBO solver

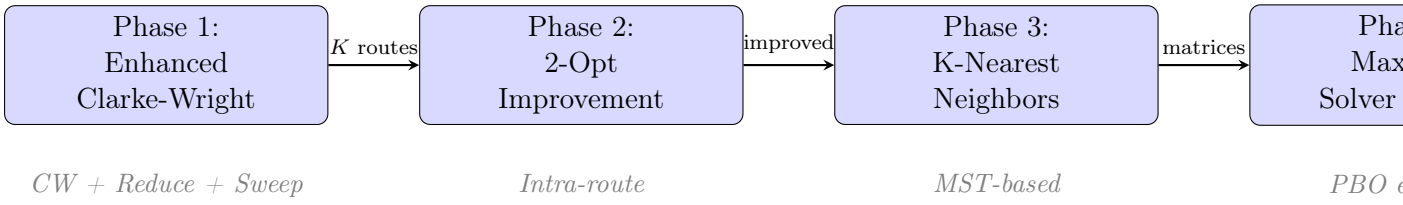

Figure 1: The four-phase pipeline of the CVRP post-improvement approach.

1.3 Key Contributions

- An **enhanced Clarke-Wright algorithm** with a four-strategy route reduction mechanism that reliably produces exactly K feasible routes, even for extremely tight capacity instances (tightness up to 100%).
- A **sweep construction fallback** that provides an alternative starting solution when the standard CW merge phase cannot reach K routes.
- A **BFD + Recursive Swap Repair** bin packing algorithm that solves the combinatorially hard route reduction sub-problem.
- Integration with a **PBO/Max-SAT solver** using MTZ subtour elimination with binary encoding of position variables.

2 Problem Formulation

2.1 Formal Definition

Definition 2.1 (Capacitated Vehicle Routing Problem)**.** Let $G = (V, E)$ be a complete undirected graph where:

- $V = \{0, 1, 2, \ldots, n-1\}$ is the set of vertices, with node 0 as the **depot**
- Nodes $\{1, 2, \ldots, n-1\}$ are **customer locations**
- Each customer i has a demand $q_i > 0$; the depot has $q_0 = 0$
- Each edge $(i, j) \in E$ has a non-negative cost d_{ij} (typically Euclidean distance)
- A fleet of K identical vehicles, each with capacity Q, is stationed at the depot

The objective is to find K routes $R_1, R_2, \ldots, R_K$ such that:

1. Each route starts and ends at the depot: $R_v = (0, c_{v,1}, c_{v,2}, \ldots, c_{v,|R_v|}, 0)$
2. Every customer is served by exactly one route: $\bigcup_{v=1}^{K} R_v = \{1, \ldots, n-1\}$ and $R_v \cap R_w = \emptyset$ for $v \neq w$
3. The capacity constraint is respected: $\sum_{c \in R_v} q_c \leq Q, \quad \forall v$
4. The total travel distance is minimized:

$$\min \sum_{v=1}^{K} \text{cost}(R_v) = \min \sum_{v=1}^{K} \left(d_{0,c_{v,1}} + \sum_{i=1}^{|R_v|-1} d_{c_{v,i},c_{v,i+1}} + d_{c_{v,|R_v|},0} \right) \tag{1}$$

2.2 Capacity Tightness

A key parameter that determines the difficulty of constructing a feasible initial solution is the **capacity tightness**:

Definition 2.2 (Capacity Tightness)**.**

$$\tau = \frac{\sum_{i=1}^{n-1} q_i}{K \cdot Q} \times 100\% \tag{2}$$

Remark 2.1. When τ approaches 100%, there is almost no slack in the capacity constraints. In such cases, the standard Clarke-Wright merge phase may fail to produce exactly K routes, since merging two routes becomes infeasible when their combined demand exceeds Q. Our enhanced reduction mechanism addresses this challenge.

3 Phase 1: Enhanced Clarke-Wright Construction

The Clarke-Wright (CW) savings algorithm is one of the most widely used heuristics for CVRP. We enhance it with a robust **four-strategy route reduction** mechanism and a **sweep construction fallback**.

3.1 Standard Clarke-Wright Algorithm

3.1.1 Savings Computation

Definition 3.1 (Savings Value)**.** For two customers $i, j \neq 0$, the savings from merging their individual routes is:

$$s_{ij} = d_{0,i} + d_{0,j} - d_{ij} \tag{3}$$

Interpretation: Instead of two separate round-trips ($0 \to i \to 0$ and $0 \to j \to 0$), merging into one route ($0 \to \cdots \to i \to j \to \cdots \to 0$) saves distance s_{ij}.

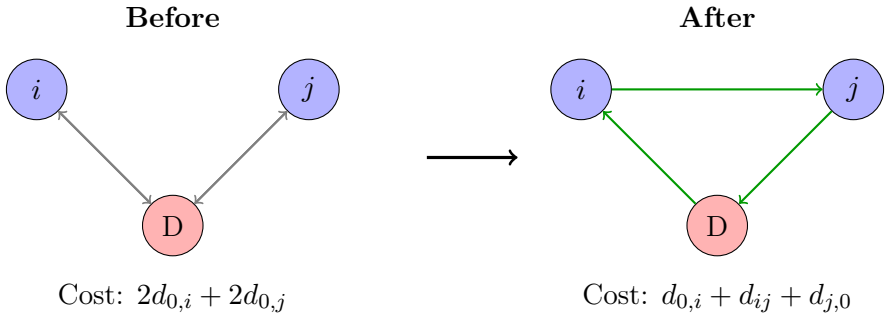

Figure 2: The savings concept: merging two individual routes saves $s_{ij} = d_{0,i} + d_{0,j} - d_{ij}$.

3.1.2 Merge Phase

The standard CW algorithm proceeds as follows:

Algorithm 1 Clarke-Wright Savings Algorithm — Standard Phase

Require: Distance matrix D, demands q, capacity Q, number of vehicles K
Ensure: Set of routes $\mathcal{R}$

Compute Savings:
for $i = 1$ to $n - 1$ **do**
  for $j = i + 1$ to $n - 1$ **do**
    $s_{ij} \leftarrow d_{0,i} + d_{0,j} - d_{ij}$
  end for
end for
Sort savings in descending order
Initialize: Create one route per customer: $R_i = [i]$ for $i = 1, \ldots, n - 1$
Merge Phase:
for each (s_{ij}, i, j) in savings list **do**
  if $R_i \neq R_j$ **and** i is at the end of R_i **and** j is at the start of R_j **then**
    if $\text{demand}(R_i) + \text{demand}(R_j) \leq Q$ **then**
      $R_i \leftarrow R_i \circ R_j$ ▷ Concatenate
      Delete R_j
    end if
  end if
end for
return $\mathcal{R}$

After the merge phase, the number of remaining routes $|\mathcal{R}|$ is typically $\geq K$. If $|\mathcal{R}| > K$, a **route reduction** phase is needed.

3.2 Four-Strategy Route Reduction

The route reduction problem is: given a feasible partition with $|\mathcal{R}| > K$ routes, transform it into a partition with exactly K routes, each respecting the capacity constraint Q.

Our method applies four strategies in sequence, from cheapest to most expensive. Each iteration reduces the route count by one.

Algorithm 2 Multi-Strategy Route Reduction

```
while |R| > K do
    if Strategy 1 (Direct Merge) succeeds then
        continue
    else if Strategy 2 (Relocate + Merge) succeeds then
        continue
    else if Strategy 3 (Dissolve Route) succeeds then
        continue
    else if Strategy 4 (Bin Packing Repack) succeeds then
        return                                   ▷ Produces exactly K routes
    else
        raise failure → trigger sweep fallback
    end if
end while
```

3.2.1 Strategy 1: Direct Merge

Find two routes whose combined demand does not exceed Q. Among all feasible pairs, choose the one with the smallest linking distance (the distance between the last customer of one route and the first customer of the other).

$$\text{Choose } (R_a, R_b) = \arg\min_{(R_i, R_j)} \left\{ \min\left(d_{R_i[-1], R_j[0]},\ d_{R_j[-1], R_i[0]}\right) \;\middle|\; \text{demand}(R_i) + \text{demand}(R_j) \leq Q \right\} \tag{4}$$

Complexity: $O(|\mathcal{R}|^2)$ per iteration.

3.2.2 Strategy 2: Relocate and Merge

When no pair of routes can be directly merged, we attempt to *move customers out* of a pair to make room for merging.

Algorithm 3 Strategy 2: Relocate and Merge

for each pair (R_a, R_b) sorted by excess demand $\delta = \text{demand}(R_a) + \text{demand}(R_b) - Q$ ascending **do**
    Collect candidates: customers from $R_a \cup R_b$, sorted by demand descending
    for each candidate customer c **do**
        if $\delta \leq 0$ **then break**
        end if
        Find a third route R_t $(t \neq a, b)$ with cheapest insertion position for c
        if $\text{demand}(R_t) + q_c \leq Q$ **then**
            Move c from its source into R_t
            $\delta \leftarrow \delta - q_c$
        end if
    end for
    if $\delta \leq 0$ **then**
        Merge R_a and R_b
        return success
    else
        Undo all moves
    end if
end for

3.2.3 Strategy 3: Dissolve Route

Dissolve the smallest route by redistributing its customers into other routes using best-insertion. Customers are placed in decreasing order of demand (largest first) to fail early if infeasible.

Algorithm 4 Strategy 3: Dissolve Route

for each route R_s in $\mathcal{R}$ sorted by size ascending **do**
    Remove R_s from $\mathcal{R}$
    Sort customers of R_s by demand descending
    placed $\leftarrow$ []
    for each customer c in R_s **do**
        Find (R_t, pos) with minimum insertion cost such that $\text{demand}(R_t) + q_c \leq Q$
        if found **then**
            Insert c at position pos in R_t
            placed.append(c, R_t)
        else
            Undo all placements; restore R_s; try next route
            break
        end if
    end for
    if all customers placed **then**
        return success
    end if
end for

3.2.4 Strategy 4: BFD + Recursive Swap Repair (Bin Packing)

When Strategies 1–3 all fail, the instance is extremely tight. We reformulate the route reduction as a **bin packing problem**: pack all $n-1$ customers into exactly K bins of capacity Q.

Greedy Phase. We first try three greedy heuristics (Balanced, First-Fit Decreasing, Best-Fit Decreasing). If any succeeds, we use that packing.

BFD + Swap Repair. If all greedy strategies fail (common when $\tau \approx 100\%$), we use a two-phase algorithm:

Algorithm 5 BFD + Recursive Swap Repair

Require: Customers sorted by demand descending, K bins of capacity Q
Ensure: Packing of all customers into K bins, or failure

Phase 1: Best-Fit Decreasing Placement
for each customer c (demand descending) **do**
    Place c in the fullest bin that still has room
    if no bin has room **then**
        Add c to `unplaced` list
    end if
end for
if `unplaced` is empty **then**
    return bins
end if
Phase 2: Recursive Swap Repair
for each c in `unplaced` **do**
    if TryPlace(c, depth $= 3$, forbidden $= \{c\}$) **then**
        Remove c from `unplaced`
    else
        return failure
    end if
end for
return bins

Algorithm 6 TryPlace — Recursive Swap

```
function TryPlace(customer c, depth, forbidden)
    d_c ← q_c                                              ▷ Demand of customer c
    for each bin b = 1, ..., K do
        if load(b) + d_c ≤ Q then
            Place c in b
            return True                                    ▷ Direct placement
        end if
    end for
    if depth = 0 then
        return False
    end if
    for each bin b do
        for each item x in bin b where x ∉ forbidden do
            if load(b) − q_x + d_c ≤ Q then
                Remove x from b; place c in b
                if TryPlace(x, depth −1, forbidden ∪{c}) then
                    return True
                end if
                Undo: remove c from b; restore x in b
            end if
        end for
    end for
    return False
end function
```

Proposition 3.1 (Complexity of Swap Repair)**.** With depth d and n customers in K bins, the worst-case complexity of TryPlace for a single unplaced item is $O((nK)^d)$. With $d = 3$, $n = 100$, and $K = 25$, this is bounded by $(2500)^3 \approx 1.6 \times 10^{10}$. In practice, early pruning and the BFD pre-placement (which places most items directly) result in sub-second runtime.

3.3 Sweep Construction Fallback

If the entire reduction phase fails (all 4 strategies exhaust), we fall back to a **sweep algorithm** to construct a completely new set of routes, then re-apply the reduction.

Algorithm 7 Sweep Construction

Require: Customers with coordinates, depot (x_0, y_0), capacity Q
Ensure: Set of routes $\mathcal{R}$

for each customer c **do**
    $\theta_c \leftarrow \text{atan2}(y_c - y_0,\ x_c - x_0)$
end for
Sort customers by θ_c
$\mathcal{R}_{\text{best}} \leftarrow \emptyset$
for $s = 0$ to $n - 2$ **do** ▷ Try all starting positions
    Build routes by sweeping from customer s, opening a new route when capacity is exceeded
    if $|\mathcal{R}| < |\mathcal{R}_{\text{best}}|$ **then**
        $\mathcal{R}_{\text{best}} \leftarrow \mathcal{R}$
    end if
end for
return $\mathcal{R}_{\text{best}}$

The sweep algorithm produces routes with more balanced loads (compared to CW's greedy merging), making it easier for the subsequent reduction to reach exactly K routes.

3.4 Overall CW + Reduction + Fallback Flow

Algorithm 8 Enhanced Clarke-Wright — Complete Procedure

Require: Instance (n, K, D, q, Q)
Ensure: K feasible routes

Run standard CW (savings → merge)
try:
    Run 4-strategy reduction to get K routes
catch failure:
    Run sweep construction
    Run 4-strategy reduction on sweep result
return K routes

3.5 Experimental Validation

We tested on 30 benchmark instances including all 23 Christofides-Mingozzi-Toth series B instances and 7 additional instances from the A, E, F, P, and X series.

Instance Group	Count	Tightness Range	Success Rate	Max Time
B-series (n31–n78)	23	82%–100%	23/23	0.003s
A, E, F, P instances	6	68%–94%	6/6	0.001s
X-n101-k25	1	100%	1/1	0.152s
Total	**30**	68%–100%	**30/30**	**0.152s**

Table 1: Enhanced CW construction results. All instances produce exactly K feasible routes.

Remark 3.1. The X-n101-k25 instance has near-zero slack ($\tau = 99.94\%$, total demand $= 5147$, $K \cdot Q = 5150$, slack $= 3$). No single greedy bin packing strategy succeeds — only the BFD + Recursive Swap Repair finds a feasible packing. This validates the necessity of Strategy 4.

4 Phase 2: 2-Opt Intra-Route Improvement

After constructing K feasible routes, we apply the classical **2-Opt** local search to improve the ordering of customers within each route independently.

4.1 2-Opt Move

A 2-opt move reverses a segment $[i, j]$ of the route. If the new route has lower cost, it replaces the current best.

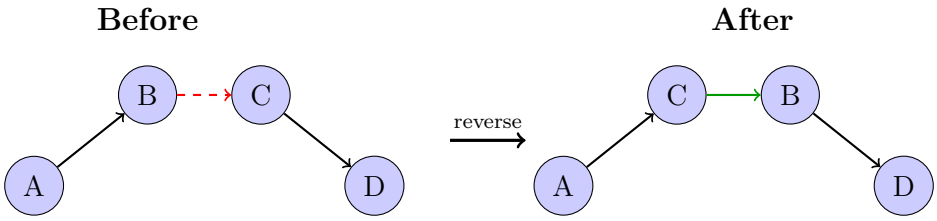

Figure 3: A 2-opt move reverses the segment between positions i and j.

Algorithm 9 2-Opt Improvement for Each Route

for each route $R_v \in \mathcal{R}$ **do**
  repeat
    improved $\leftarrow$ False
    for $i = 0$ to $|R_v| - 2$ **do**
      for $j = i + 1$ to $|R_v| - 1$ **do**
        $R' \leftarrow$ reverse segment $[i, j]$ in R_v
        if cost(R') < cost(R_v) **then**
          $R_v \leftarrow R'$; improved $\leftarrow$ True
        end if
      end for
    end for
  until NOT improved
end for

Complexity: $O(|R_v|^2)$ per improvement pass, repeated until convergence.

5 Phase 3: K-Nearest Neighbors Expansion

The 2-opt-improved solution is a local optimum. To allow the Max-SAT solver to explore beyond this local optimum, we construct **restricted distance matrices** for each vehicle, including:

1. All edges in the current route
2. For each customer in the route, its K nearest neighbors (using MST and distance matrix)

5.1 MST-Based Neighbor Selection

We first compute a Minimum Spanning Tree (MST) of the full distance graph. Neighbors of customer c in the MST are sorted by edge weight. If the MST provides fewer than K neighbors, we fill the remaining slots from the distance matrix.

Algorithm 10 K-Nearest Neighbors Matrix Construction

Require: Instance, current routes $\mathcal{R}$, parameter K
Ensure: Distance matrices $\{M_v\}$ for each vehicle v

Compute MST of the full distance graph
for each route R_v **do**
    Initialize M_v with -1 (all edges invalid)
    Set $M_v[i, i] = 0$ for all i
                                ▷ Include current route edges
    for each consecutive pair (c_i, c_{i+1}) in R_v (including depot links) **do**
        $M_v[c_i, c_{i+1}] \leftarrow d_{c_i, c_{i+1}}$
        $M_v[c_{i+1}, c_i] \leftarrow d_{c_{i+1}, c_i}$
    end for
                                ▷ Include K-nearest neighbor edges
    for each customer c in R_v **do**
        neighbors $\leftarrow$ top-K from MST $\cup$ distance matrix
        for each $c' \in$ neighbors **do**
            $M_v[c, c'] \leftarrow d_{c,c'}$
        end for
    end for
end for

Key Insight: Edges marked as -1 in M_v are *forbidden* for vehicle v. This restricts the Max-SAT solver's search space from $O(n^2)$ to $O(K \cdot n)$ edges per vehicle, making the problem tractable.

6 Phase 4: Max-SAT Global Optimization

The core optimization engine formulates CVRP as a **Weighted Partial Max-SAT** (equivalently, Pseudo-Boolean Optimization) problem and solves it using the `clasp` solver.

6.1 Boolean Variable Encoding

We define three types of Boolean variables:

Variable	Type	Meaning
$w_{i,j,v}$	Edge	Vehicle v travels directly from node i to node j
$t_{i,v}$	Assignment	Customer i is assigned to vehicle v
$u_{i,b,v}$	Position (bit b)	Bit b of the position of customer i on vehicle v

Table 2: Boolean variables for the CVRP encoding.

The position variable $u_{i,v}$ is encoded in binary with $\lceil \log_2(n-1) \rceil$ bits:

$$u_{i,v} = \sum_{b=0}^{B-1} 2^b \cdot u_{i,b,v}, \quad B = \lceil \log_2(n-1) \rceil \tag{5}$$

6.2 Hard Constraints

6.2.1 Depot Constraints

Each vehicle leaves and returns to the depot exactly once:

$$\sum_{j=1}^{n-1} w_{0,j,v} = 1, \quad \forall v \tag{C1}$$

$$\sum_{i=1}^{n-1} w_{i,0,v} = 1, \quad \forall v \tag{C2}$$

6.2.2 Customer Visit Constraints

Each customer is visited exactly once by exactly one vehicle:

$$\sum_{v=1}^{K} \sum_{\substack{j=0 \\ j \neq i}}^{n-1} w_{i,j,v} = 1, \quad \forall i \in \{1, \ldots, n-1\} \tag{C3: one exit}$$

$$\sum_{v=1}^{K} \sum_{\substack{i=0 \\ i \neq j}}^{n-1} w_{i,j,v} = 1, \quad \forall j \in \{1, \ldots, n-1\} \tag{C4: one entry}$$

6.2.3 No Back-and-Forth

$$\neg w_{i,j,v} \vee \neg w_{j,i,v}, \quad \forall i < j, \ \forall v \tag{C5}$$

6.2.4 Vehicle-Customer Assignment

If an edge on vehicle v involves customer i, then i is assigned to v:

$$w_{i,j,v} \Rightarrow t_{i,v} \quad \text{and} \quad w_{i,j,v} \Rightarrow t_{j,v}, \quad \forall i, j \geq 1, \ i \neq j \tag{C6–C7}$$

$$w_{0,j,v} \Rightarrow t_{j,v} \quad \text{and} \quad w_{i,0,v} \Rightarrow t_{i,v} \tag{C8–C9}$$

6.2.5 Exclusive Assignment

Each customer belongs to at most one vehicle:

$$\neg t_{i,v} \vee \neg t_{i,v'}, \quad \forall i \geq 1, \ \forall v \neq v' \tag{C10}$$

6.2.6 Subtour Elimination (MTZ with Binary Encoding)

The Miller-Tucker-Zemlin (MTZ) constraints prevent subtours using position variables:

$$u_{i,v} - u_{j,v} + (n-1) \cdot w_{i,j,v} \leq n-2, \quad \forall i,j \in \{1,\ldots,n-1\},\ i \neq j,\ \forall v \tag{C11}$$

Interpretation: If vehicle v travels from i to j ($w_{i,j,v} = 1$), then $u_{j,v} \geq u_{i,v} + 1$, establishing a strict ordering that prevents cycles.

In the binary encoding, this becomes a Pseudo-Boolean constraint:

$$\sum_{b=0}^{B-1} 2^b \cdot (-u_{i,b,v} + u_{j,b,v}) + (-(n-1)) \cdot w_{i,j,v} \geq -(n-2) \tag{6}$$

6.2.7 Capacity Constraint

$$\sum_{i=1}^{n-1} q_i \cdot t_{i,v} \leq Q, \quad \forall v \tag{C12}$$

Equivalently: $\sum_{i=0}^{n-1} (-q_i) \cdot t_{i,v} \geq -Q$

6.2.8 Search Space Restriction

Edges not in the K-neighbors matrix are forced to zero:

$$w_{i,j,v} = 0, \quad \forall (i,j,v) \text{ where } M_v[i,j] = -1 \tag{C13}$$

6.3 Objective Function (Soft Clauses)

The objective minimizes total travel distance:

$$\min \sum_{v=1}^{K} \sum_{i=0}^{n-1} \sum_{\substack{j=0 \\ j \neq i}}^{n-1} d_{i,j} \cdot w_{i,j,v} \tag{7}$$

In PBO format, this is encoded as:

$$\texttt{min:} \quad \sum_{i,j,v} d_{i,j} \cdot x_{w_{i,j,v}}$$

6.4 Model Statistics

Component	**Count**	**Asymptotic**
Edge variables $w_{i,j,v}$	$K \cdot n \cdot (n-1)$	$O(Kn^2)$
Assignment variables $t_{i,v}$	$K \cdot n$	$O(Kn)$
Position bits $u_{i,b,v}$	$K \cdot (n-1) \cdot \lceil \log_2(n-1) \rceil$	$O(Kn \log n)$
Total variables		$O(Kn^2)$
Total constraints		$O(Kn^2 \log n)$

Table 3: Model size as a function of n (number of nodes) and K (number of vehicles).

6.5 Alternative: Lima Reachability Formulation

The solver also supports an alternative subtour elimination method based on **reachability variables** (Lima formulation), inspired by the RTSS approach:

- Define $c_{i,j,v} = 1$ if customer i is reachable before customer j on vehicle v
- **Base:** $w_{i,j,v} \Rightarrow c_{i,j,v}$
- **Induction:** $w_{i,j,v} \wedge c_{j,k,v} \Rightarrow c_{i,k,v}$
- **Acyclic:** $c_{i,i,v} = 0$ for all i

This avoids integer position variables at the cost of $O(n^3)$ transitivity constraints.

6.6 Solver Invocation

The model is written in OPB (Pseudo-Boolean) format and solved by the `clasp` PBO solver with a configurable time limit:

Listing 1: Solver invocation

```
def solve(self):
    with open('input.txt', 'w+') as f:
        f.write(self.encode())

    system('./clasp input.txt > output.txt --time-limit=80')

    with open('output.txt', 'r') as f:
        self.decode(f.readlines())
```

7 Complete Pipeline

7.1 Entry Point

Listing 2: Main pipeline (run.py)

```
cvrp = Instance(argv[1]).load()

# Phase 1: Enhanced Clarke-Wright
cw_time, routes = ClarkeWright.run(cvrp, int(argv[2]))

# Phase 2: 2-Opt
to_time, routes = TwoOpt.run(routes)

cw_cost = sum(route.cost for route in routes.values())
print(f'CW + 2-Opt cost: {cw_cost} ({cw_time + to_time:.3f}s)')

# Phase 3: K-Nearest Neighbors
_, matrices = KNeighbors.run(cvrp, int(argv[3]), routes)

# Phase 4: Max-SAT Solver
```

```
solver_time, solver_cost, solver_routes = Solver.run(cvrp, matrices)

print(f'Solver cost: {solver_cost} ({solver_time:.3f}s)')
print(f'Improvement: {(cw_cost - solver_cost) / cw_cost * 100:.2f}%')
```

7.2 Data Flow

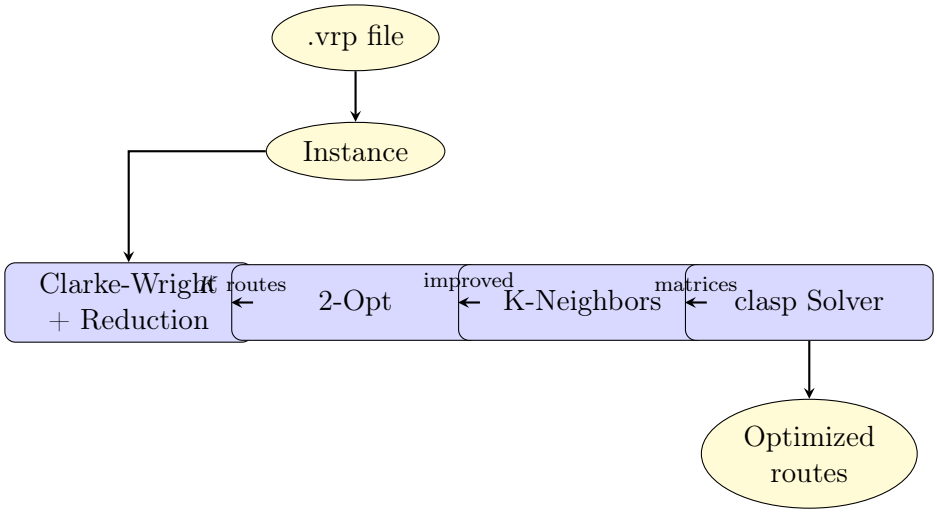

Figure 4: Data flow through the pipeline.

8 Implementation Architecture

8.1 Module Overview

Module	Responsibility
`classes/instance.py`	Parse VRP files (supports EUC_2D, ATT, EXPLICIT distance types); compute distance matrix
`classes/route.py`	Route data structure with lazy demand/cost caching
`classes/clarke_wright.py`	Enhanced CW algorithm with 4-strategy reduction and sweep fallback
`classes/two_opt.py`	Intra-route 2-opt improvement
`classes/k_neighbors.py`	MST-based K-nearest neighbor matrix construction
`classes/solver.py`	PBO encoding and `clasp` solver interface
`classes/utils.py`	Timer decorator and utility functions
`run.py`	Main entry point for the 4-phase pipeline

Table 4: Project modules and their responsibilities.

8.2 Route Data Structure

The `Route` class provides lazy caching for demand and cost:

Listing 3: Route class with lazy caching

```python
class Route:
    def __init__(self, cvrp, value, demand=-1):
        self.cvrp = cvrp
        self.value = value   # List of customer indices
        self._demand = demand  # Cached demand (-1 = recompute)
        self._cost = -1        # Cached cost (-1 = recompute)

    @property
    def demand(self):
        if self._demand < 0:
            self._demand = sum(self.cvrp.demands[c] for c in self.
                value)
        return self._demand

    @property
    def cost(self):
        if self._cost < 0:
            self._cost = self.cvrp.distances[0, self.value[0]]
            for i in range(len(self.value) - 1):
                self._cost += self.cvrp.distances[self.value[i],
                    self.value[i+1]]
            self._cost += self.cvrp.distances[self.value[-1], 0]
        return self._cost
```

8.3 Instance Format

The solver reads standard TSPLIB-format `.vrp` files:

```
NAME : B-n31-k5
COMMENT : (Augerat, 1995)
TYPE : CVRP
DIMENSION : 31
EDGE_WEIGHT_TYPE : EUC_2D
CAPACITY : 100
NODE_COORD_SECTION
 1  82  76
 2  96  44
 ...
DEMAND_SECTION
 1  0
 2  19
 ...
DEPOT_SECTION
 1
 -1
```

`EOF`

Supported distance types: `EUC_2D` (Euclidean), `ATT` (pseudo-Euclidean), `EXPLICIT` (given matrix in `LOWER_ROW` or `LOWER_COL` format).

9 Theoretical Analysis

9.1 Correctness of Route Reduction

Theorem 9.1 (Soundness)**.** Every route set produced by the enhanced CW algorithm is a feasible CVRP solution: each customer is served exactly once, each route starts and ends at the depot, and every route respects the capacity constraint Q.

Proof. 1. **Customer coverage:** The initial routes contain every customer. All strategies only move customers between routes or reassign them to bins — no customer is ever dropped.

2. **Capacity:** Every merge (Strategy 1, 2) checks $\text{demand}(R_a) + \text{demand}(R_b) \leq Q$ before merging. Every insertion (Strategy 3) checks $\text{demand}(R_t) + q_c \leq Q$ before inserting. Strategy 4 checks $\text{load}(b) + q_c \leq Q$ before placement.

3. **Undo safety:** Strategies 2 and 3 perform full rollback on failure, restoring the original route configuration.

□

Theorem 9.2 (Completeness of Max-SAT Model)**.** If a feasible improvement to the current solution exists within the K-neighbors search space, the `clasp` solver will find the optimal such improvement.

Proof. The PBO model correctly encodes all CVRP constraints (Hamiltonian routes with capacity limits). The search space is complete within the edges defined by the K-neighbors matrices. The `clasp` solver is an exact PBO solver that finds the optimum of the encoded problem (subject to the time limit). □

9.2 Complexity Summary

Phase	**Time Complexity**	**Typical Runtime**
CW: Savings computation	$O(n^2)$	< 0.001s
CW: Merge phase	$O(n^2 \log n)$	< 0.001s
CW: Route reduction	$O(n^3)$ (Strategies 1–3)	< 0.01s
CW: Bin packing (Strategy 4)	$O((nK)^d)$ worst case	< 0.2s
2-Opt	$O(\sum_v \lvert R_v \rvert^3)$	< 0.1s
K-Neighbors	$O(n^2 \log n)$ (MST)	< 0.5s
Max-SAT (clasp)	NP-hard	≤ 80s (time limit)

Table 5: Complexity and typical runtime per phase.

10 Usage

10.1 Requirements

- Python $\geq$ 3.10 (uses `match` statements)
- `numpy` — distance matrices and demand arrays
- `networkx` — minimum spanning tree computation
- `clasp` — PBO solver binary (must be in the project root)

10.2 Running the Pipeline

```
# Install dependencies
pip install -r requirements.txt

# Run the full pipeline
# Usage: python run.py <instance> <vehicle_number> <neighbor_number>

python run.py instances/A-n33-k6.vrp 6 5
python run.py instances/B/B-n45-k6.vrp 6 5
python run.py X-n101-k25.vrp 25 5
```

10.3 Output

```
CW + 2-Opt cost: 846 (0.001s)
CW + 2-Opt routes: [Route[...], Route[...], ...]
Solver cost: 780 (45.2s)
Solver routes: [w_0_5_0, w_5_12_0, ...]
Improvement: 7.80%
```

11 Conclusion

This document presented a complete post-improvement pipeline for CVRP that combines constructive heuristics with exact optimization:

1. **Enhanced Clarke-Wright** with four-strategy route reduction (direct merge, relocate-and-merge, dissolve, BFD+swap) and sweep fallback ensures reliable initial solution construction for all instance types, including those with 100% capacity tightness.
2. **2-Opt local search** quickly improves the initial route orderings.
3. **K-Nearest Neighbors** using MST defines a tractable search space for the exact solver.
4. **Max-SAT/PBO solver** (`clasp`) performs global optimization within the restricted search space, using MTZ subtour elimination with binary position encoding.

The key algorithmic contribution is the **BFD + Recursive Swap Repair** mechanism in Strategy 4, which solves the NP-hard bin packing sub-problem arising in extremely tight instances. By combining BFD pre-placement with bounded-depth recursive swaps, the algorithm achieves sub-second performance while finding feasible packings that no greedy heuristic can produce.